

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE

INGENIERÍA DE REQUERIMIENTOS (ISR-401)

PRÁCTICA EXPERIMENTAL
UNIDAD IV

*Inspección, gestión del cambio, trazabilidad CASE
y línea base del ERS FabroGym*

INTEGRANTES:

CASTRO BAJAÑA ARIEL OMAR
MERA ARIAS ERICK JHAIR
SANCHEZ CENTENO ROSELYN ANDREINA

DOCENTE:

PhD. Guerrero Ulloa Gleiston Cicerón

FECHA:

05 de agosto de 2026

REPOSITORIO PÚBLICO

último commit el 05/08/26 a las 23h00
<https://github.com/Emeraxs/ISR401-PFC-ERS.git>

QUEVEDO – ECUADOR
2026

Índice

1	Resumen y objetivos	3
2	Fundamento teórico	3
2.1	Marco normativo	3
2.2	Validación de requisitos	4
2.2.1	Verificación frente a validación	4
2.2.2	Principios y técnicas de validación	4
2.2.3	El método de inspección Fagan	5
2.2.4	Métricas de inspección	5
2.2.5	Caso ilustrativo	5
2.3	Gestión de requisitos	6
2.3.1	Atributos de los requisitos	6
2.3.2	Trazabilidad: pre-traceability, post-traceability y matriz	6
2.3.3	El proceso de cambio: RFC, impacto, CCB, cierre	7
2.3.4	Métricas de volatilidad	7
2.4	Herramientas CASE para IR	7
2.4.1	Clasificación	7
2.4.2	Funcionalidades esperables	7
2.4.3	Criterios de selección, limitaciones y costos	8
2.5	IR en procesos ágiles	8
2.5.1	Backlog, historias de usuario y criterios de aceptación	8
2.5.2	Grooming y Definition of Ready / Done	8
2.5.3	BDD y Gherkin	8
2.5.4	Trazabilidad en entornos ágiles	8
2.5.5	Contraste entre IR documental e IR ágil	9
3	Inspección Fagan	10
3.1	Roles y alcance	10
3.2	Preparación individual	10
3.3	Acta de la reunión	10
3.4	Registro de defectos	10
3.5	Métricas de inspección	10
4	Correcciones aplicadas	10
5	Gestión del cambio y CCB	10
5.1	RFC-01	10
5.2	RFC-02	10
5.3	RFC-03	10
5.4	Acta del CCB	10
6	Trazabilidad en herramienta CASE	10
6.1	Estructura del tablero	10
6.2	Matriz de trazabilidad	10
6.3	Evidencia	10
7	Línea base con Git	10
8	Comparativa de herramientas CASE	10
9	IR tradicional frente a IR ágil	10

10 Conclusiones críticas	10
11 Distribución del trabajo	10
12 Referencias	10
13 Anexos	11

1 Resumen y objetivos

Objetivo general

Aplicar técnicas formales de validación de requisitos, gestionar cambios mediante un Change Control Board simulado, implementar la trazabilidad del ERS en una herramienta CASE, establecer una línea base con Git y comparar metodológicamente la IR tradicional frente a la IR ágil, sobre el ERS real del Proyecto Fin de Curso.

Objetivos específicos

1. Ejecutar una inspección formal tipo Fagan sobre el ERS del PFC, con roles diferenciados, lista de verificación y registro de defectos.
2. Calcular e interpretar métricas de inspección (densidad de defectos, distribución por tipo y severidad, tasa de corrección).
3. Tramitar tres solicitudes formales de cambio (RFC) con análisis de impacto sustentado en la matriz de trazabilidad.
4. Deliberar y decidir en un CCB simulado, dejando acta motivada y versión actualizada del ERS.
5. Construir la trazabilidad bidireccional del ERS en una herramienta CASE de gestión (Jira o Trello) con campos personalizados y enlaces vericables.
6. Establecer y publicar la línea base del ERS con control de versiones (commit semántico, tag anotado y CHANGELOG.md).
7. Comparar herramientas CASE de IR y contrastar IR tradicional frente a IR ágil, cerrando con conclusiones críticas propias.

2 Fundamento teórico

2.1 Marco normativo

La validación y la gestión de requisitos no son prácticas aisladas: están formalmente ubicadas dentro de los procesos normalizados de ingeniería de sistemas y software. La norma ISO/IEC/IEEE 29148:2018 define el proceso de Ingeniería de Requisitos y, dentro de él, la actividad de *validar requisitos* en el apartado 6.4, cuyo propósito es confirmar que el conjunto de requisitos especificados constituye una representación correcta y completa de las necesidades del stakeholder, y que cada requisito es verificable frente a un método de comprobación definido [1]. Esta actividad se distingue de la especificación (6.2) y del análisis (6.3), que la preceden en el ciclo de vida del requisito.

A nivel de procesos de ciclo de vida más amplios, la norma ISO/IEC/IEEE 15288:2023 — orientada a sistemas — y su contraparte ISO/IEC/IEEE 12207:2017 — orientada a software — ubican el control de cambios dentro del proceso de *Gestión de la Configuración*, cuyo objetivo es establecer y mantener la integridad de los elementos de configuración (entre ellos, el ERS) a lo largo de su evolución [2, 3]. Todo cambio a un requisito ya baselineado debe, según estas normas, pasar por un proceso controlado de solicitud, evaluación de impacto, aprobación o rechazo, y registro, lo cual es precisamente lo que un Change Control Board (CCB) opera en la práctica.

El modelo de calidad de producto de la norma ISO/IEC 25010:2023 [4] es el referente para juzgar la calidad de los requisitos no funcionales del ERS: sus características (adecuación

funcional, eficiencia de desempeño, compatibilidad, usabilidad, confiabilidad, seguridad, mantenibilidad y portabilidad) proporcionan el vocabulario con el cual un NFR puede evaluarse como completo, medible y verificable, y no como una aspiración genérica.

El SWEBOK v4.0, en su Área de Conocimiento 1 (Requisitos de Software), subapartados 1.6 a 1.8, consolida estas prácticas en tres bloques: validación de requisitos, gestión de requisitos, y herramientas de ingeniería de requisitos, presentándolos como continuación lógica de la elicitación y el análisis [5]. En la misma línea, Pohl [6], en la Parte V de su obra, desarrolla en profundidad la validación mediante inspecciones formales y la gestión del cambio como disciplinas complementarias: la primera detecta defectos antes de que el ERS se congele; la segunda gestiona su evolución controlada después de la línea base.

Finalmente, el Change Control Board como mecanismo de gobierno de cambios está descrito en el PMBOK, 7.^a edición [7], dentro de las prácticas de gestión de la configuración y el alcance: un CCB es un grupo formalmente constituido, responsable de revisar, evaluar, aprobar, retrasar o rechazar cambios al proyecto, y de registrar y comunicar sus decisiones. Aplicado a un ERS, el CCB reemplaza la aprobación informal de cambios por una deliberación trazable y auditable.

2.2 Validación de requisitos

2.2.1 Verificación frente a validación

La distinción clásica — y frecuentemente confundida en la práctica — es que la **verificación** responde a la pregunta “¿estamos construyendo el producto correctamente?”, es decir, si el requisito cumple criterios internos de calidad (no ambiguo, consistente, verificable), mientras que la **validación** responde a “¿estamos construyendo el producto correcto?”, es decir, si el conjunto de requisitos representa fielmente lo que el stakeholder necesita [1, 6]. Un requisito puede estar perfectamente verificado (bien redactado, medible) y, sin embargo, no ser válido porque no responde a una necesidad real del negocio.

2.2.2 Principios y técnicas de validación

Pohl [6] plantea que la validación debe ser continua —no un evento único al final— e involucrar activamente a los stakeholders, no solo al equipo técnico. Entre las técnicas disponibles se encuentran:

- **Revisión (review):** lectura crítica del documento por personas distintas del autor, sin protocolo estricto de roles.
- **Walkthrough:** el autor guía a los revisores a través del documento, útil para socializar el contenido más que para encontrar defectos sistemáticamente.
- **Inspección formal (Fagan):** técnica estructurada, con roles diferenciados, checklist y métricas, descrita en detalle a continuación.
- **Prototipado:** construcción de una representación ejecutable parcial del sistema para validar requisitos difíciles de verificar solo por lectura (p. ej., interacción de usuario).
- **Listas de verificación (checklists):** conjuntos de preguntas orientadas a propiedades específicas del requisito (ambigüedad, completitud, consistencia, verificabilidad, trazabilidad, factibilidad), tal como las exige el Anexo A de la ISO/IEC/IEEE 29148:2018.
- **Perspectivas (perspective-based reading):** cada revisor examina el documento desde un rol distinto (usuario, desarrollador, probador), lo que amplía el tipo de defectos detectados.

2.2.3 El método de inspección Fagan

La inspección formal fue introducida por Fagan [8] como un proceso estructurado para detectar defectos en artefactos de desarrollo de software mediante revisión por pares, con el objetivo de reducir errores en etapas tempranas, cuando su corrección es más económica. El método define cinco fases:

1. **Planificación:** el moderador selecciona el material a inspeccionar, asigna los roles y programa la reunión.
2. **Overview (visión general):** el autor presenta el contexto del artefacto a los inspectores.
3. **Preparación individual:** cada inspector examina el material de forma independiente y registra los defectos que encuentra, antes de la reunión conjunta.
4. **Reunión de inspección:** conducida por el moderador, con el lector parafraseando el contenido sección por sección y los inspectores reportando los defectos encontrados; el propósito es *detectar*, no discutir soluciones.
5. **Reelaboración y seguimiento:** el autor corrige los defectos y el moderador verifica que las correcciones sean adecuadas antes de cerrar el ciclo.

Fagan [8] distingue cuatro roles: el *moderador* (conduce el proceso y consolida el registro), el *lector* (parafrasea el contenido sin leerlo literalmente, para forzar comprensión en vez de lectura mecánica), y los *inspectores* (detectan defectos desde su perspectiva particular). Este reparto de roles es precisamente el que exige el Anexo A y el Paso 1 de la presente práctica.

2.2.4 Métricas de inspección

De un proceso de inspección se derivan métricas cuantitativas que permiten interpretar la calidad del artefacto revisado y la eficacia del propio proceso [8, 6]:

- **Densidad de defectos:** número de defectos encontrados por página o por requisito inspeccionado; un valor alto sugiere que el ERS necesita una revisión más profunda antes de avanzar.
- **Distribución por tipo:** clasifica los defectos según la propiedad violada (ambigüedad, omisión, inconsistencia, etc.), lo que orienta dónde reforzar la elicitación o la redacción.
- **Distribución por severidad:** separa defectos críticos, mayores y menores, priorizando el esfuerzo de corrección.
- **Tasa de detección por inspector:** compara la efectividad individual, útil para calibrar checklists o capacitación futura.
- **Esfuerzo total (horas-persona):** permite calcular el costo del proceso de inspección y contrastarlo contra el costo estimado de corregir esos mismos defectos en etapas posteriores del proyecto.

2.2.5 Caso ilustrativo

Durante la inspección del ERS del sistema de gestión administrativa y operativa para FabroGym se detectó una inconsistencia entre dos requisitos relacionados con la administración de membresías. El requisito **RF-MEM-02** establece que el sistema debe permitir activar o renovar una membresía únicamente cuando el pago correspondiente haya sido registrado y validado. Sin embargo, en el flujo descrito para el proceso de pagos se observó que el requisito **RF-PAG-02**

contempla la posibilidad de mantener pagos pendientes, permitiendo posteriormente su regularización.

La revisión evidenció que el documento no especificaba con claridad si un cliente con un pago pendiente podía conservar una membresía activa o si esta debía permanecer suspendida hasta completar el pago. Esta situación originaba dos interpretaciones distintas para un mismo proceso de negocio, lo que fue clasificado durante la inspección Fagan como un defecto de **inconsistencia**.

Como acción correctiva, el equipo acordó modificar el ERS para establecer que la activación o renovación de la membresía solo podrá completarse cuando el estado del pago sea *confirmado*. Mientras exista un pago pendiente, la membresía permanecerá en estado *pendiente de activación*, evitando así ambigüedades entre los módulos de membresías y pagos.

Este caso demuestra cómo la aplicación del método de inspección Fagan permite identificar contradicciones entre requisitos funcionales antes de la etapa de implementación, mejorando la consistencia interna del ERS y reduciendo el riesgo de retrabajo durante el desarrollo del sistema [6, 5].

2.3 Gestión de requisitos

subsubsectionGestión de la configuración, línea base y versionado

La gestión de requisitos comprende el conjunto de actividades orientadas a mantener la integridad, la trazabilidad y la evolución controlada del conjunto de requisitos a lo largo del proyecto [1, 2]. Un elemento central es la **gestión de la configuración**: el ERS se trata como un elemento de configuración sujeto a control de versiones, de modo que cada cambio quede identificado, registrado y sea reversible. La **línea base (baseline)** es el punto de referencia formal — una versión del ERS aprobada y congelada — a partir del cual cualquier modificación posterior debe tramitarse como un cambio controlado y no como una edición libre [3, 6]. En términos prácticos de control de versiones, esto se traduce en un *commit* semántico, un *tag* anotado y un registro de cambios (`CHANGELOG.md`) que documenten qué contiene cada versión y por qué evolucionó.

2.3.1 Atributos de los requisitos

Wiegers y Beatty [9] recomiendan asociar a cada requisito un conjunto de atributos que lo hacen gestionable como un artefacto individual, entre ellos: identificador único, estado (propuesto, aprobado, implementado, verificado), prioridad, origen o fuente, estabilidad, criticidad de negocio y método de verificación asociado. Estos atributos son los que después permiten construir campos personalizados en una herramienta CASE y priorizar objetivamente frente a un cambio propuesto.

2.3.2 Trazabilidad: pre-traceability, post-traceability y matriz

Gotel y Finkelstein [10] caracterizan el problema de la trazabilidad de requisitos distinguiendo dos fases: la **pre-traceability**, que documenta el origen de un requisito antes de su especificación (de qué stakeholder o necesidad de negocio surge), y la **post-traceability**, que documenta hacia dónde se proyecta un requisito una vez especificado (casos de uso, componentes de diseño, código, casos de prueba). Cleland-Huang, Gotel y Zisman [11] amplían este marco distinguiendo la trazabilidad *hacia atrás* (backward, del requisito a su origen) y *hacia adelante* (forward, del requisito a sus artefactos derivados), y señalan que la trazabilidad bidireccional — exigir ambos sentidos para cada requisito — es la que realmente sostiene el análisis de impacto ante un cambio.

La **matriz de trazabilidad** es el artefacto que materializa esta relación en forma tabular, típicamente en la forma Stakeholder → Requisito Funcional → Caso de Uso → Componente/Clase → Caso de Prueba [9, 11]. Un requisito sin enlaces — huérfano — en cualquiera de los dos sentidos es, en sí mismo, un defecto de gestión que debe reportarse.

2.3.3 El proceso de cambio: RFC, impacto, CCB, cierre

Una vez establecida la línea base, todo cambio debe tramitarse mediante una **solicitud formal de cambio** (Request for Change, RFC), que documenta el requisito afectado, la justificación de negocio y el cambio propuesto [7, 3]. El **análisis de impacto** recorre sistemáticamente la matriz de trazabilidad para identificar tanto los artefactos directamente afectados como los indirectos (por ejemplo, un cambio en un requisito funcional puede repercutir en un requisito no funcional de rendimiento asociado). El **CCB** delibera sobre la RFC con base en ese análisis y emite una decisión — aprobado, aprobado con condiciones, rechazado o diferido— que se registra en un acta motivada [7]. Finalmente, la **implementación** traslada la decisión al ERS (nueva versión, nueva línea base) y el **cierre** verifica que el cambio fue correctamente aplicado y comunicado.

2.3.4 Métricas de volatilidad

La volatilidad de requisitos mide cuánto cambia el conjunto de requisitos respecto a una línea base, típicamente como proporción de requisitos añadidos, modificados o eliminados en un periodo dado [6, 9]. Una volatilidad alta y sostenida suele indicar elicitación insuficiente en las etapas tempranas, mientras que picos puntuales asociados a RFC concretas son un indicador normal de la evolución controlada del proyecto.

2.4 Herramientas CASE para IR

2.4.1 Clasificación

Las herramientas CASE (Computer-Aided Software Engineering) se clasifican tradicionalmente en tres categorías [5, 6]:

- **Upper CASE:** orientadas a las fases tempranas del ciclo de vida — elicitación, análisis, modelado y especificación de requisitos—, como los gestores de requisitos con soporte de trazabilidad.
- **Lower CASE:** orientadas a las fases posteriores — implementación, generación de código, pruebas—.
- **Integrated CASE (I-CASE):** cubren el ciclo de vida completo, integrando repositorio, modelado, gestión de cambios y generación de artefactos en una sola plataforma.

Herramientas como Jira o Trello, usadas con tableros de backlog y campos personalizados, funcionan en la práctica como herramientas upper CASE ligeras para gestión de requisitos, mientras que suites como IBM DOORS o Jama Connect corresponden más estrictamente a herramientas I-CASE especializadas en requisitos con trazabilidad nativa.

2.4.2 Funcionalidades esperables

Para que una herramienta CASE soporte adecuadamente la gestión de requisitos, el SWEBOK v4.0 [5] y Pohl [6] coinciden en que debe ofrecer, como mínimo: un **repositorio** centralizado de requisitos; soporte para **atributos** personalizados por requisito; **trazabilidad** bidireccional enlazable; capacidad de **baselining** (fijar y comparar versiones); **control de cambios** integrado (flujos de aprobación); soporte de **colaboración** entre stakeholders distribuidos; e **integración con sistemas de control de versiones** (VCS) para sincronizar el estado del requisito con el del código o la documentación asociada.

2.4.3 Criterios de selección, limitaciones y costos

La selección de una herramienta CASE debe considerar la cobertura funcional real frente a las necesidades del proyecto, la escalabilidad para equipos y volúmenes de requisitos crecientes, la facilidad de integración con el resto del ecosistema de desarrollo (VCS, CI/CD), el costo de licenciamiento (herramientas como Jira o Trello ofrecen planes gratuitos limitados, mientras que DOORS o Jama Connect son soluciones empresariales de costo considerablemente mayor) y la curva de aprendizaje del equipo. Entre las limitaciones más comunes de las herramientas livianas orientadas a tableros ágiles (Jira, Trello) está la ausencia nativa de trazabilidad formal hacia atrás (a los stakeholders o documentos de origen), la cual debe suplirse mediante campos personalizados y disciplina del equipo, a diferencia de las herramientas especializadas en requisitos, que la incorporan como funcionalidad central [6].

2.5 IR en procesos ágiles

2.5.1 Backlog, historias de usuario y criterios de aceptación

En los enfoques ágiles, la especificación de requisitos se organiza alrededor del **product backlog**: una lista priorizada y viva de todo lo que el producto podría necesitar, en contraste con el ERS documental que se congela en una línea base [12]. Cada elemento del backlog suele expresarse como **historia de usuario**, con el formato “Como *[rol]*, quiero *[funcionalidad]*, para *[beneficio]*”, que captura la intención del stakeholder sin fijar prematuramente los detalles de implementación [12]. Cada historia se completa con **criterios de aceptación**: condiciones verificables que determinan cuándo la historia se considera correctamente implementada, y que cumplen, en el contexto ágil, un papel equivalente al de los requisitos verificables del enfoque documental.

2.5.2 Grooming y Definition of Ready / Done

El **refinamiento del backlog** (grooming o backlog refinement) es la actividad recurrente en la que el equipo revisa, detalla, estima y reordena las historias antes de que entren a un sprint [12]. Dos acuerdos de equipo delimitan la calidad de este proceso: la **Definition of Ready (DoR)**, que establece las condiciones mínimas que una historia debe cumplir para poder ser planificada (criterios de aceptación claros, dependencias identificadas, estimación posible), y la **Definition of Done (DoD)**, que establece las condiciones para considerar una historia terminada (código probado, criterios de aceptación verificados, documentación actualizada). Ambas funcionan como un control de calidad equivalente, en espíritu, a las listas de verificación de la inspección formal, aunque aplicadas de forma continua en vez de puntual.

2.5.3 BDD y Gherkin

El desarrollo guiado por comportamiento (*Behavior-Driven Development*, BDD) propone escribir los criterios de aceptación en un lenguaje estructurado y semiformal — el formato **Gherkin**, con la estructura **Dado / Cuando / Entonces** (*Given / When / Then*) — de modo que el mismo artefacto sirva simultáneamente como especificación legible por el stakeholder y como prueba automatizable por el equipo técnico [13]. Este mecanismo reduce la brecha entre la validación documental (revisar que el requisito esté bien escrito) y la validación ejecutable (comprobar que el sistema efectivamente lo cumple).

2.5.4 Trazabilidad en entornos ágiles

En un entorno ágil, la trazabilidad no desaparece, pero cambia de soporte: en lugar de una matriz estática vinculada a un ERS congelado, se sostiene mediante enlaces entre epics, historias, sub-tareas y pruebas dentro de la propia herramienta de gestión del backlog (por ejemplo, Jira o

Trello), lo cual exige la misma disciplina bidireccional — hacia el origen y hacia los artefactos derivados— descrita para el enfoque documental [11, 12].

2.5.5 Contraste entre IR documental e IR ágil

Frente al enfoque documental — que privilegia un ERS exhaustivo, validado formalmente y controlado mediante un CCB antes de cualquier cambio—, el enfoque ágil privilegia la entrega incremental de valor y la adaptación continua del backlog a partir de la retroalimentación del stakeholder [12, 5]. Esto no implica ausencia de disciplina: implica que el control de calidad y el control de cambios se distribuyen en ciclos cortos (sprint a sprint) en lugar de concentrarse en hitos formales. El primer enfoque resulta más adecuado en contextos con requisitos regulatorios o contractuales estrictos, alta necesidad de auditoría y bajo margen de tolerancia a errores tardíos (por ejemplo, sistemas críticos o de cumplimiento normativo); el segundo resulta más adecuado en contextos de alta incertidumbre de producto, stakeholders disponibles de forma continua y necesidad de retroalimentación temprana del usuario final.

3 Inspección Fagan

3.1 Roles y alcance

3.2 Preparación individual

3.3 Acta de la reunión

3.4 Registro de defectos

3.5 Métricas de inspección

4 Correcciones aplicadas

5 Gestión del cambio y CCB

5.1 RFC-01

5.2 RFC-02

5.3 RFC-03

5.4 Acta del CCB

6 Trazabilidad en herramienta CASE

6.1 Estructura del tablero

6.2 Matriz de trazabilidad

6.3 Evidencia

7 Línea base con Git

8 Comparativa de herramientas CASE

9 IR tradicional frente a IR ágil

10 Conclusiones críticas

- 1.
- 2.
- 3.
- 4.
- 5.

11 Distribución del trabajo

12 Referencias

Referencias

- [1] ISO/IEC/IEEE, “29148:2018 – systems and software engineering: Life cycle processes: Requirements engineering,” ISO/IEC/IEEE, Std., 2018, doi: 10.1109/IEEESTD.2018.8559686.

- [2] —, “15288:2023 – systems and software engineering: System life cycle processes,” ISO/IEC/IEEE, Std., 2023.
- [3] —, “12207:2017 – systems and software engineering: Software life cycle processes,” ISO/IEC/IEEE, Std., 2017.
- [4] ISO/IEC, “25010:2023 – systems and software quality requirements and evaluation (SQuaRE): Product quality model,” ISO/IEC, Std., 2023.
- [5] H. Washizaki, Ed., *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide)*, version 4.0 ed. Los Alamitos, CA, USA: IEEE Computer Society, 2024.
- [6] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*, 2nd ed. Berlin, Germany: Springer, 2025, doi: 10.1007/978-3-662-69204-2.
- [7] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Newtown Square, PA, USA: PMI, 2021.
- [8] M. E. Fagan, “Design and code inspections to reduce errors in program development,” *IBM Systems Journal*, vol. 15, no. 3, pp. 182–211, 1976, doi: 10.1147/sj.153.0182.
- [9] K. Wiegers and J. Beatty, *Software Requirements*, 3rd ed. Redmond, WA, USA: Microsoft Press, 2013.
- [10] O. C. Z. Gotel and A. C. W. Finkelstein, “An analysis of the requirements traceability problem,” in *Proc. IEEE Int. Conf. Requirements Engineering (ICRE)*, Colorado Springs, CO, USA, 1994, pp. 94–101, doi: 10.1109/ICRE.1994.292398.
- [11] J. Cleland-Huang, O. Gotel, and A. Zisman, Eds., *Software and Systems Traceability*. London, U.K.: Springer, 2012, doi: 10.1007/978-1-4471-2239-5.
- [12] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA, USA: Addison-Wesley, 2004.
- [13] J. F. Smart, *BDD in Action: Behavior-Driven Development for the Whole Software Lifecycle*. Shelter Island, NY, USA: Manning Publications, 2014.

13 Anexos

Anexo A – Lista de verificación de inspección

Anexo B – Registro de defectos

Anexo C – Formularios RFC y Acta del CCB

Anexo D – Exportación CSV del backlog

Anexo E – Capturas del tablero CASE y del repositorio Git

Anexo F – Referencias IEEE y declaración de uso de IA